Assignment 7

Week 7

6733172621 Patthadon Phengpinij
Collaborators. Claude (for LaTeX styling and grammar checking)

Problem 1. Z-Gate

Use hand calculation and co-lab to show that the Z-gate converts $|-\rangle$ to $|+\rangle$.

Solution

Hand calculation. The Z-gate is $Z = \sigma_z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$: it leaves $|0\rangle$ alone and flips the sign of $|1\rangle$. With $|-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$,

$$Z|-\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \begin{pmatrix} 1 \\ -1 \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix} = |+\rangle .$$

In Dirac notation the same step reads

$$Z|-\rangle = \tfrac{1}{\sqrt{2}}(Z|0\rangle - Z|1\rangle) = \tfrac{1}{\sqrt{2}}(|0\rangle - (-|1\rangle)) = \tfrac{1}{\sqrt{2}}(|0\rangle + |1\rangle) = |+\rangle .$$

By the same token $Z|+\rangle = |-\rangle$: in the $|+\rangle / |-\rangle$ basis the Z-gate acts as a NOT gate, which is the key to **Problem 2**.

Colab.

```
Z     = np.array([[1, 0], [0, -1]])
plus  = np.array([1,  1]) / np.sqrt(2)
minus = np.array([1, -1]) / np.sqrt(2)

print(Z @ minus)
print(np.allclose(Z @ minus, plus), np.allclose(Z @ plus, minus))
```

The notebook (**Appendix 1**) gives $Z|-\rangle = (0.7071, 0.7071)^\top = |+\rangle$ and `True` for both checks, confirming the hand calculation.

Problem 2. Controlled-Z Gate

Construct a controlled-Z gate from the controlled phase shift gate. Then show that it is a CNOT gate in the $|+\rangle / |-\rangle$ basis.

Solution

Controlled phase shift. The phase shift gate $P(\varphi) = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\varphi} \end{pmatrix}$ multiplies $|1\rangle$ by $e^{i\varphi}$. Its controlled version applies $P(\varphi)$ to the target only when the control (MSB) is $|1\rangle$:

$$CP(\varphi) = |0\rangle\langle 0| \otimes I + |1\rangle\langle 1| \otimes P(\varphi) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & e^{i\varphi} \end{pmatrix}.$$

Controlled-Z. Choose $\varphi = \pi$. Since $e^{i\pi} = -1$, the phase shift becomes $P(\pi) = \mathrm{diag}(1, -1) = Z$, and

$$CZ = CP(\pi) = |0\rangle\langle 0| \otimes I + |1\rangle\langle 1| \otimes Z = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix}.$$

Only $|11\rangle$ picks up a sign. Because that sign needs *both* qubits to be 1, CZ is symmetric: it does not matter which qubit is called the control.

CNOT in the $|+\rangle / |-\rangle$ basis. Keep the control in the $|0\rangle / |1\rangle$ basis and write the target in the $|+\rangle / |-\rangle$ basis. When the control is $|0\rangle$ nothing happens; when it is $|1\rangle$ the target gets Z, which exchanges $|+\rangle \leftrightarrow |-\rangle$ (**Problem 1**):

CZ, target in $\|+\rangle / \|-\rangle$	CNOT, target in $\|0\rangle / \|1\rangle$
$\|0\rangle \|+\rangle \mapsto \|0\rangle \|+\rangle$	$\|0\rangle \|0\rangle \mapsto \|0\rangle \|0\rangle$
$\|0\rangle \|-\rangle \mapsto \|0\rangle \|-\rangle$	$\|0\rangle \|1\rangle \mapsto \|0\rangle \|1\rangle$
$\|1\rangle \|+\rangle \mapsto \|1\rangle \|-\rangle$	$\|1\rangle \|0\rangle \mapsto \|1\rangle \|1\rangle$
$\|1\rangle \|-\rangle \mapsto \|1\rangle \|+\rangle$	$\|1\rangle \|1\rangle \mapsto \|1\rangle \|0\rangle$

The two tables are the same once $|+\rangle$ is read as "0" and $|-\rangle$ as "1": the target flips exactly when the control is 1.

As a matrix. The new basis $\{|0+\rangle, |0-\rangle, |1+\rangle, |1-\rangle\}$ is obtained from the old one by $I \otimes H$ (since $H|0\rangle = |+\rangle$ and $H|1\rangle = |-\rangle$), and $H = H^\dagger = H^{-1}$. So the matrix of CZ in the new basis is

$$\begin{aligned}(I \otimes H)\ CZ\ (I \otimes H) &= |0\rangle\langle 0| \otimes HIH + |1\rangle\langle 1| \otimes HZH \\ &= |0\rangle\langle 0| \otimes I + |1\rangle\langle 1| \otimes X = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} = \text{CNOT},\end{aligned}$$

using $HIH = H^2 = I$ and

$$HZH = \frac{1}{2}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}\begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} = \frac{1}{2}\begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} = \frac{1}{2}\begin{pmatrix} 0 & 2 \\ 2 & 0 \end{pmatrix} = X.$$

Hence CZ is a CNOT gate in the $|+\rangle / |-\rangle$ basis. ■ The notebook (**Appendix 1**) confirms $CP(\pi) = \text{diag}(1, 1, 1, -1)$, $(I \otimes H)\ CZ\ (I \otimes H) = \text{CNOT}$, and $CZ|1+\rangle = |1-\rangle$.

Problem 3. Implementation of a SWAP gate

A SWAP gate can be implemented using 3 CNOT gates in sequence. We will discuss this later. But try it now to challenge yourselves. The first and the third CNOT gates have the first bit as the control bit, but the second CNOT gate has the second qubit as the control bit. Try to create the matrix of the second CNOT gate and then prove the circuit is indeed a SWAP gate.

Solution

Conventions. Write two-qubit states as $|a\,b\rangle$, with a the first bit (MSB) and b the second (LSB), in the order $|00\rangle, |01\rangle, |10\rangle, |11\rangle$. Call the CNOT with the first bit as control C_M and the one with the second bit as control C_L:

$$C_M : |a\,b\rangle \mapsto |a,\ a \oplus b\rangle, \qquad C_L : |a\,b\rangle \mapsto |a \oplus b,\ b\rangle.$$

Matrix of the second CNOT. The j-th column of a gate's matrix is the image of the j-th basis state. For C_L the control is the LSB, so only $|01\rangle$ and $|11\rangle$ get their MSB flipped:

$$C_L|00\rangle = |00\rangle, \quad C_L|01\rangle = |11\rangle, \quad C_L|10\rangle = |10\rangle, \quad C_L|11\rangle = |01\rangle.$$

Placing these images as columns, and comparing with the first-bit-controlled CNOT,

$$C_L = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}, \qquad C_M = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}.$$

So C_L exchanges $|01\rangle \leftrightarrow |11\rangle$, while C_M exchanges $|10\rangle \leftrightarrow |11\rangle$.

The circuit is a SWAP. The gates act in the order C_M, C_L, C_M, so the circuit's matrix is the product with the first gate on the right, $U = C_M C_L C_M$. The first two gates give

$$C_L C_M = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{pmatrix},$$

and the third C_M, multiplied on the left, exchanges rows 3 and 4:

$$U = C_M\,(C_L C_M) = \begin{pmatrix}1&0&0&0\\0&1&0&0\\0&0&0&1\\0&0&1&0\end{pmatrix}\begin{pmatrix}1&0&0&0\\0&0&1&0\\0&0&0&1\\0&1&0&0\end{pmatrix} = \begin{pmatrix}1&0&0&0\\0&0&1&0\\0&1&0&0\\0&0&0&1\end{pmatrix} = U_{\text{SWAP}}.$$

This matrix fixes $|00\rangle$ and $|11\rangle$ and exchanges $|01\rangle \leftrightarrow |10\rangle$, i.e. $U\,|a\,b\rangle = |b\,a\rangle$. On a general state,

$$U\,(\alpha\,|00\rangle + \beta\,|01\rangle + \gamma\,|10\rangle + \delta\,|11\rangle) = \alpha\,|00\rangle + \gamma\,|01\rangle + \beta\,|10\rangle + \delta\,|11\rangle\,,$$

so the two qubits trade places, exactly as the state labels in **Figure 2** show. The circuit is indeed a SWAP gate. ■ The notebook (**Appendix 1**) builds C_M, C_L and SWAP from their truth tables and confirms $C_M C_L C_M = U_{\text{SWAP}}$.

Problem 4. 1-qubit Hadamard Gate

Show Equation below:

$$\begin{aligned}\boldsymbol{H} &= \frac{1}{\sqrt{2}}\begin{pmatrix}1&1\\1&-1\end{pmatrix}\begin{pmatrix}\alpha\\\beta\end{pmatrix}\\ &= \frac{1}{\sqrt{2}}\begin{pmatrix}\alpha+\beta\\\alpha-\beta\end{pmatrix}\\ \boldsymbol{H} &= \alpha\,|+\rangle + \beta\,|-\rangle\end{aligned}$$

by substituting $|+\rangle$ and $|-\rangle$ in terms of $|0\rangle$ and $|1\rangle$.

Solution

Let $|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle = (\alpha,\beta)^\top$; the equation describes the Hadamard gate acting on this state, so its left-hand side is $H\,|\psi\rangle$.
Matrix action.

$$H\,|\psi\rangle = \frac{1}{\sqrt{2}}\begin{pmatrix}1&1\\1&-1\end{pmatrix}\begin{pmatrix}\alpha\\\beta\end{pmatrix} = \frac{1}{\sqrt{2}}\begin{pmatrix}\alpha+\beta\\\alpha-\beta\end{pmatrix} = \frac{\alpha+\beta}{\sqrt{2}}\,|0\rangle + \frac{\alpha-\beta}{\sqrt{2}}\,|1\rangle\,.$$

Substituting $|\pm\rangle$. With $|+\rangle = \frac{1}{\sqrt{2}}\,(|0\rangle + |1\rangle)$ and $|-\rangle = \frac{1}{\sqrt{2}}\,(|0\rangle - |1\rangle)$,

$$\begin{aligned}\alpha\,|+\rangle + \beta\,|-\rangle &= \frac{\alpha}{\sqrt{2}}\,(|0\rangle + |1\rangle) + \frac{\beta}{\sqrt{2}}\,(|0\rangle - |1\rangle)\\ &= \frac{\alpha+\beta}{\sqrt{2}}\,|0\rangle + \frac{\alpha-\beta}{\sqrt{2}}\,|1\rangle = \frac{1}{\sqrt{2}}\begin{pmatrix}\alpha+\beta\\\alpha-\beta\end{pmatrix}.\end{aligned}$$

The two results agree coefficient by coefficient, so

$$H\,|\psi\rangle = \frac{1}{\sqrt{2}}\begin{pmatrix}\alpha+\beta\\\alpha-\beta\end{pmatrix} = \alpha\,|+\rangle + \beta\,|-\rangle\,. \qquad \blacksquare$$

In words: H sends $|0\rangle \mapsto |+\rangle$ and $|1\rangle \mapsto |-\rangle$, and by linearity it carries the amplitudes α, β over unchanged – the same coordinates, now in the $|+\rangle\,/\,|-\rangle$ basis. The notebook (**Appendix 1**) checks the identity symbolically with SymPy.

Problem 5. *n*-**qubit Hadamard Gate Numerical Substitution 1**

Use Equation below to compute $\boldsymbol{H}^{\otimes 2}|2\rangle$. What is n in this case?

$$\boldsymbol{H}^{\otimes n}|y\rangle = \frac{1}{2^{n/2}}\sum_{x=0}^{2^n-1}(-1)^{x\cdot y}|x\rangle$$

Solution

Reading off n and y. The operator $H^{\otimes 2}$ acts on a register of $\mathbf{n = 2}$ qubits, and $|2\rangle$ is a basis state of that register: $2 = 10_2$, so $y = (y_1, y_0) = (1, 0)$. The sum runs over $x = 0, \ldots, 2^n - 1 = 3$ with prefactor $1/2^{n/2} = 1/2$.

The sign. $x \cdot y$ is the bitwise inner product taken mod 2, $x \cdot y = x_1y_1 \oplus x_0y_0 = x_1$, since $y = 10_2$:

x	x_1x_0	$x \cdot y = x_1y_1 \oplus x_0y_0$	$(-1)^{x\cdot y}$	term
0	00	$0\cdot 1 \oplus 0\cdot 0 = 0$	$+1$	$+\,\|00\rangle$
1	01	$0\cdot 1 \oplus 1\cdot 0 = 0$	$+1$	$+\,\|01\rangle$
2	10	$1\cdot 1 \oplus 0\cdot 0 = 1$	-1	$-\,\|10\rangle$
3	11	$1\cdot 1 \oplus 1\cdot 0 = 1$	-1	$-\,\|11\rangle$

Result.

$$H^{\otimes 2}|2\rangle = \frac{1}{2}\sum_{x=0}^{3}(-1)^{x\cdot 2}|x\rangle = \frac{1}{2}\left(|0\rangle + |1\rangle - |2\rangle - |3\rangle\right) = \frac{1}{2}\left(|00\rangle + |01\rangle - |10\rangle - |11\rangle\right),$$

which is normalized: $4 \times \left(\frac{1}{2}\right)^2 = 1$.

Problem 6. *n*-**qubit Hadamard Gate Numerical Substitution 2**

Use expansion method in Equation below:

$$\begin{aligned}
\boldsymbol{H}^{\otimes n}|y\rangle &= \boldsymbol{H}^{\otimes n}|y_{n-1}y_{n-2}\ldots y_1y_0\rangle \\
&= (\boldsymbol{H}|y_{n-1}\rangle)\otimes(\boldsymbol{H}|y_{n-2}\rangle)\otimes\cdots\otimes(\boldsymbol{H}|y_0\rangle) \\
&= \left(\frac{1}{\sqrt{2}}\left(|0\rangle + (-1)^{y_{n-1}}|1\rangle\right)\right)\otimes\left(\frac{1}{\sqrt{2}}\left(|0\rangle + (-1)^{y_{n-2}}|1\rangle\right)\right)\otimes\cdots\otimes\left(\frac{1}{\sqrt{2}}\left(|0\rangle + (-1)^{y_0}|1\rangle\right)\right) \\
\boldsymbol{H}^{\otimes n}|y\rangle &= \frac{1}{2^{n/2}}\left(|0\rangle + (-1)^{y_{n-1}}|1\rangle\right)\otimes\left(|0\rangle + (-1)^{y_{n-2}}|1\rangle\right)\otimes\cdots\otimes\left(|0\rangle + (-1)^{y_0}|1\rangle\right)
\end{aligned}$$

to compute $\boldsymbol{H}^{\otimes 2}|2\rangle$. Try to identify the fact that only when x_i and y_i are both 1 would it get a contribution of -1 to the final coefficient.

Solution

Expansion. With $n = 2$ and $|y\rangle = |2\rangle = |y_1y_0\rangle = |10\rangle$, there is one factor per qubit:

$$\begin{aligned}
H^{\otimes 2}|10\rangle = (H|1\rangle)\otimes(H|0\rangle) &= \frac{1}{2}\left(|0\rangle + (-1)^1|1\rangle\right)\otimes\left(|0\rangle + (-1)^0|1\rangle\right) \\
&= \frac{1}{2}\left(|0\rangle - |1\rangle\right)\otimes\left(|0\rangle + |1\rangle\right) = \frac{1}{2}\left(|00\rangle + |01\rangle - |10\rangle - |11\rangle\right),
\end{aligned}$$

the same state as in **Problem 5**.

Where the minus signs come from. Expanding the product, the term $|x_1x_0\rangle$ is built by picking $|x_1\rangle$ from the first factor and $|x_0\rangle$ from the second. Picking $|0\rangle$ from factor i contributes $+1$; picking $|1\rangle$ contributes $(-1)^{y_i}$. Either way factor i contributes $(-1)^{x_iy_i}$, which is -1 *only* when $x_i = y_i = 1$:

$\|x_1x_0\rangle$	qubit 1: $(-1)^{x_1y_1}$, $y_1 = 1$	qubit 0: $(-1)^{x_0y_0}$, $y_0 = 0$	coefficient $\times 2$
$\|00\rangle$	$+1$	$+1$	$+1$
$\|01\rangle$	$+1$	$+1$	$+1$
$\|10\rangle$	-1 $(x_1 = y_1 = 1)$	$+1$	-1
$\|11\rangle$	-1 $(x_1 = y_1 = 1)$	$+1$	-1

Since $y_0 = 0$, the second factor $(|0\rangle + |1\rangle)$ never produces a sign; every minus sign comes from the one position where both x_1 and y_1 are 1. Multiplying the factors,

$$\text{coefficient of } |x\rangle = \frac{1}{2^{n/2}} \prod_i (-1)^{x_i y_i} = \frac{1}{2^{n/2}} (-1)^{\sum_i x_i y_i} = \frac{1}{2^{n/2}} (-1)^{x \cdot y},$$

which recovers the formula of **Problem 5**. Only the parity of the number of positions where both bits are 1 matters, which is why $x \cdot y$ may be taken mod 2.

Problem 7. **n-qubit Hadamard Gate Numerical Substitution 3**

Compute $\boldsymbol{H}^{\otimes 2} |2\rangle$ but use matrix multiplications.

Solution

The matrix of $H^{\otimes 2}$. Each entry of H multiplies a full copy of H:

$$H \otimes H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \cdot H & 1 \cdot H \\ 1 \cdot H & -1 \cdot H \end{pmatrix} = \frac{1}{2} \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{pmatrix}.$$

Acting on $|2\rangle$. In the ordered basis $|00\rangle, |01\rangle, |10\rangle, |11\rangle$, $|2\rangle = |10\rangle = (0,0,1,0)^\top$, so the product simply picks out the third column:

$$H^{\otimes 2} |2\rangle = \frac{1}{2} \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{pmatrix} \begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \end{pmatrix} = \frac{1}{2} \begin{pmatrix} 1 \\ 1 \\ -1 \\ -1 \end{pmatrix} = \frac{1}{2} \left(|00\rangle + |01\rangle - |10\rangle - |11\rangle \right).$$

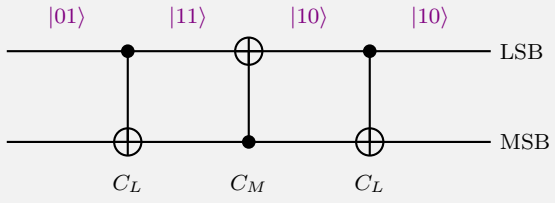
All three methods (**Problems 5–7**) agree. In fact the matrix *is* the formula of **Problem 5** written out as a table: its entry in row x, column y is $\frac{1}{2}(-1)^{x \cdot y}$. For example, row 01, column 01 has $x \cdot y = 1$, giving $-\frac{1}{2}$; row 11, column 11 has $x \cdot y = 1 \oplus 1 = 0$, giving $+\frac{1}{2}$. The notebook (**Appendix 1**) evaluates all three methods and finds the same vector $\frac{1}{2}(1,1,-1,-1)^\top$.

Problem 8. **SWAP Gate Implementation**

We may change the implementation of the SWAP gate using LSB-controlling CNOT gate followed by MSB-controlling CNOT gate and then followed by LSBcontrolling CNOT gate. Please repeat what we have done in the main text and prove that using the intuition method and matrix multiplication.

Solution

With the notation of **Problem 3** ($|a\,b\rangle$ with a the MSB; C_L: the LSB controls and the MSB is flipped; C_M: the MSB controls and the LSB is flipped), the new circuit applies C_L, then C_M, then C_L. Drawn like **Figure 2**, with an example basis state on top:



Intuition method: following the bits. Each CNOT XORs its control bit into its target bit, so for a basis state $|a\,b\rangle$

$$\begin{aligned} |a\,b\rangle &\xrightarrow{C_L} |a \oplus b,\ b\rangle \\ &\xrightarrow{C_M} |a \oplus b,\ b \oplus (a \oplus b)\rangle = |a \oplus b,\ a\rangle \end{aligned}$$

$$\xrightarrow{C_L} |(a \oplus b) \oplus a,\ a\rangle = |b\, a\rangle\,,$$

using $x \oplus x = 0$. The two bits end up exchanged. The example on top of the circuit is the case $(a, b) = (0, 1)$: $|01\rangle \to |11\rangle \to |10\rangle \to |10\rangle$.

Intuition method: following the amplitudes. Every gate is linear, so it is enough to know which basis states it exchanges: C_L swaps $|01\rangle \leftrightarrow |11\rangle$ and C_M swaps $|10\rangle \leftrightarrow |11\rangle$. Starting from $|\Psi\rangle = \alpha\,|00\rangle + \beta\,|01\rangle + \gamma\,|10\rangle + \delta\,|11\rangle$,

$$\begin{aligned}
|\Phi_1\rangle &= C_L\,|\Psi\rangle = \alpha\,|00\rangle + \delta\,|01\rangle + \gamma\,|10\rangle + \beta\,|11\rangle\,,\\
|\Phi_2\rangle &= C_M\,|\Phi_1\rangle = \alpha\,|00\rangle + \delta\,|01\rangle + \beta\,|10\rangle + \gamma\,|11\rangle\,,\\
|\Phi\rangle &= C_L\,|\Phi_2\rangle = \alpha\,|00\rangle + \gamma\,|01\rangle + \beta\,|10\rangle + \delta\,|11\rangle\,,
\end{aligned}$$

so β and γ trade places while α and δ stay put: $|\Phi\rangle = U_{\text{SWAP}}\,|\Psi\rangle$.

Matrix multiplication. The circuit's matrix is now $U' = C_L C_M C_L$. The first two gates give

$$C_M C_L = \begin{pmatrix}1&0&0&0\\0&1&0&0\\0&0&0&1\\0&0&1&0\end{pmatrix}\begin{pmatrix}1&0&0&0\\0&0&0&1\\0&0&1&0\\0&1&0&0\end{pmatrix} = \begin{pmatrix}1&0&0&0\\0&0&0&1\\0&1&0&0\\0&0&1&0\end{pmatrix},$$

and the last C_L, multiplied on the left, exchanges rows 2 and 4 (the rows of $|01\rangle$ and $|11\rangle$):

$$U' = C_L\,(C_M C_L) = \begin{pmatrix}1&0&0&0\\0&0&0&1\\0&0&1&0\\0&1&0&0\end{pmatrix}\begin{pmatrix}1&0&0&0\\0&0&0&1\\0&1&0&0\\0&0&1&0\end{pmatrix} = \begin{pmatrix}1&0&0&0\\0&0&1&0\\0&1&0&0\\0&0&0&1\end{pmatrix} = U_{\text{SWAP}}. \qquad \blacksquare$$

> **Remark.** Both orderings work because SWAP is symmetric in the two qubits. Relabelling MSB $\leftrightarrow$ LSB turns C_M into C_L and vice versa ($C_L = U_{\text{SWAP}}\,C_M\,U_{\text{SWAP}}$, see **Problem 9**), so this circuit is the circuit of **Problem 3** drawn upside down – and a gate that exchanges the two qubits cannot tell which way up it is drawn.

The notebook (**Appendix 1**) confirms $C_L C_M C_L = U_{\text{SWAP}}$ and traces all four basis states through the circuit.

Problem 9. LSB-Controlling CNOT Gate

Repeat the definition and derivation of the CNOT gate matrix in **Chap.14** to obtain the matrix of LSB-controlling CNOT gate. That is to derive $U_{\text{XOR,center}}$ in the following equation:

$$\begin{aligned}
U_{\text{XOR,right}}U_{\text{XOR,center}}U_{\text{XOR,left}} &= \begin{pmatrix}1&0&0&0\\0&1&0&0\\0&0&0&1\\0&0&1&0\end{pmatrix}\begin{pmatrix}1&0&0&0\\0&0&0&1\\0&0&1&0\\0&1&0&0\end{pmatrix}\begin{pmatrix}1&0&0&0\\0&1&0&0\\0&0&0&1\\0&0&1&0\end{pmatrix}\\
&= \begin{pmatrix}1&0&0&0\\0&0&0&1\\0&1&0&0\\0&0&1&0\end{pmatrix}\begin{pmatrix}1&0&0&0\\0&0&0&1\\0&0&1&0\\0&1&0&0\end{pmatrix}\\
&= \begin{pmatrix}1&0&0&0\\0&0&1&0\\0&1&0&0\\0&0&0&1\end{pmatrix}\\
U_{\text{XOR,right}}U_{\text{XOR,center}}U_{\text{XOR,left}} &= U_{\text{SWAP}}
\end{aligned}$$

Solution

Definition. A CNOT (XOR) gate writes the XOR of its control and target bits into the target and leaves the control unchanged. For the LSB-controlling gate the LSB b is the control and the

MSB a is the target, so on $|a\,b\rangle$

$$U_{\text{XOR,center}}\,|a\,b\rangle = |a \oplus b,\ b\rangle\,.$$

Truth table.

input $\lvert a\,b\rangle$	control b	new MSB $a \oplus b$	output $\lvert a \oplus b,\ b\rangle$
$\lvert 00\rangle$	0	0	$\lvert 00\rangle$
$\lvert 01\rangle$	1	1	$\lvert 11\rangle$
$\lvert 10\rangle$	0	1	$\lvert 10\rangle$
$\lvert 11\rangle$	1	0	$\lvert 01\rangle$

Building the matrix. A gate that sends each basis input $|j\rangle$ to a basis output $|f(j)\rangle$ is $U = \sum_j |f(j)\rangle\langle j|$; indeed $U|k\rangle = \sum_j |f(j)\rangle\langle j|k\rangle = |f(k)\rangle$. Reading the truth table row by row,

$$U_{\text{XOR,center}} = |00\rangle\langle 00| + |11\rangle\langle 01| + |10\rangle\langle 10| + |01\rangle\langle 11|\,.$$

Each outer product $|i\rangle\langle j|$ is the matrix with a single 1 in row i, column j (rows and columns ordered $00, 01, 10, 11$), so the four 1s sit at $(00, 00)$, $(11, 01)$, $(10, 10)$, and $(01, 11)$:

$$U_{\text{XOR,center}} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix},$$

the middle matrix of the equation, and the same C_L found column by column in **Problem 3**.
Compact form and a cross-check. Grouping the terms by the value of the control (the LSB),

$$U_{\text{XOR,center}} = I \otimes |0\rangle\langle 0| + X \otimes |1\rangle\langle 1|\,, \qquad \text{vs.} \qquad U_{\text{XOR,left}} = U_{\text{XOR,right}} = |0\rangle\langle 0| \otimes I + |1\rangle\langle 1| \otimes X,$$

the mirror image of the MSB-controlling gate. Mirroring the two qubits is exactly what SWAP does, and indeed

$$U_{\text{SWAP}}\,U_{\text{XOR,left}}\,U_{\text{SWAP}} : |a\,b\rangle \mapsto |b\,a\rangle \mapsto |b,\ b \oplus a\rangle \mapsto |a \oplus b,\ b\rangle = U_{\text{XOR,center}}\,|a\,b\rangle\,.$$

The full equation. $U_{\text{XOR,left}} = U_{\text{XOR,right}}$ swaps $|10\rangle \leftrightarrow |11\rangle$, so multiplying by it on the left exchanges rows 3 and 4 and multiplying on the right exchanges columns 3 and 4:

$$U_{\text{XOR,right}}\,U_{\text{XOR,center}} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix} \quad \text{(rows 3, 4 of } U_{\text{XOR,center}} \text{ exchanged)},$$

$$(U_{\text{XOR,right}}\,U_{\text{XOR,center}})\,U_{\text{XOR,left}} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} = U_{\text{SWAP}} \quad \text{(columns 3, 4 exchanged)},$$

reproducing every line of the given equation. ■ The notebook (**Appendix 1**) builds $U_{\text{XOR,center}}$ from the outer-product sum and checks it against both closed forms and the full product.

Problem 10. Quantum Computing Exercise 1

Run the basis transformation circuit in **Figure 1** on IBM-Q using real hardware and simulation. Do necessary modification to change the input to $|11\rangle$ and check the output.

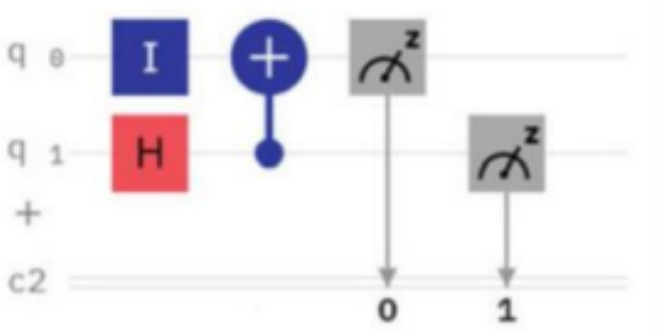


Figure 1: The Quantum Circuit

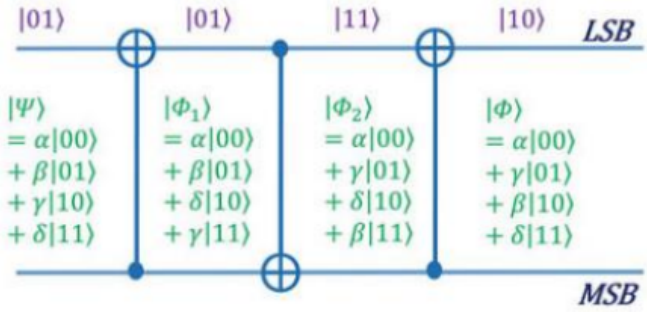


Figure 2: Implementation of a SWAP gate using CNOT gates. The purple ket's on top shows an example of how a single basis vector evolves in the circuit.

Solution

What the circuit does. In **Figure 1** the MSB q_1 gets a Hadamard and then controls a CNOT onto q_0; the I on q_0 is an identity placeholder. For an input $|a\,b\rangle$ (MSB first),

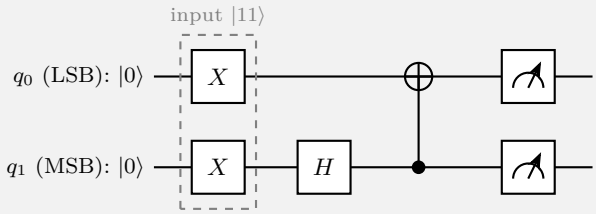
$$|a\,b\rangle \xrightarrow{H\otimes I} \tfrac{1}{\sqrt{2}}\left(|0\rangle + (-1)^a\,|1\rangle\right)\otimes|b\rangle$$
$$\xrightarrow{\text{CNOT}} \tfrac{1}{\sqrt{2}}\left(|0,\,b\rangle + (-1)^a\,|1,\,1\oplus b\rangle\right),$$

so the circuit maps the computational basis onto the four Bell states – hence "basis transformation":

input	output	measured bit strings
$\|00\rangle$	$\|\phi^+\rangle = \frac{1}{\sqrt{2}}(\|00\rangle + \|11\rangle)$	`00`, `11` (50% each)
$\|01\rangle$	$\|\psi^+\rangle = \frac{1}{\sqrt{2}}(\|01\rangle + \|10\rangle)$	`01`, `10` (50% each)
$\|10\rangle$	$\|\phi^-\rangle = \frac{1}{\sqrt{2}}(\|00\rangle - \|11\rangle)$	`00`, `11` (50% each)
$\|11\rangle$	$\|\psi^-\rangle = \frac{1}{\sqrt{2}}(\|01\rangle - \|10\rangle)$	`01`, `10` (50% each)

Run as given, the composer starts from $|00\rangle$, so both the simulator and the hardware should show two bars, `00` and `11`, at about 50% each.

Changing the input to $|11\rangle$. The composer always starts in $|00\rangle$, so $|11\rangle$ has to be prepared with a NOT on each qubit: replace the I on q_0 with X, and put an X before the H on q_1.



Expected output. Step by step, MSB first,

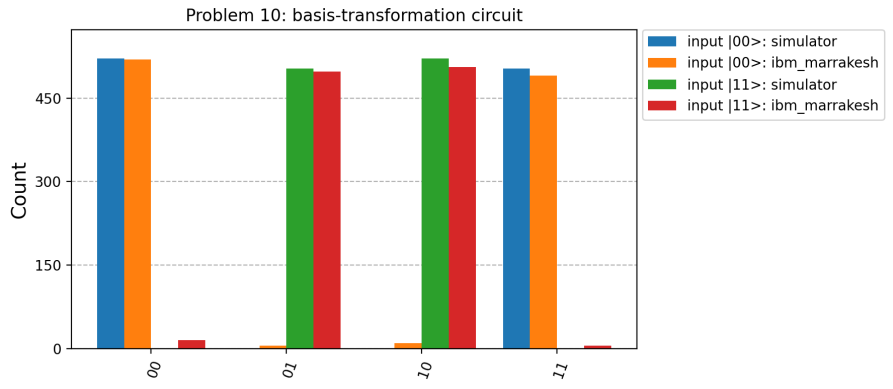
$$(X \otimes X)\,|00\rangle = |11\rangle\,,$$
$$(H \otimes I)\,|11\rangle = \tfrac{1}{\sqrt{2}}\,(|0\rangle - |1\rangle) \otimes |1\rangle = \tfrac{1}{\sqrt{2}}\,(|01\rangle - |11\rangle)\,,$$
$$\text{CNOT}\,\tfrac{1}{\sqrt{2}}\,(|01\rangle - |11\rangle) = \tfrac{1}{\sqrt{2}}\,(|01\rangle - |10\rangle) = |\psi^-\rangle\,,$$

so $\mathrm{P}(\texttt{01}) = \mathrm{P}(\texttt{10}) = \frac{1}{2}$ and $\mathrm{P}(\texttt{00}) = \mathrm{P}(\texttt{11}) = 0$.

Running it. Both versions were built in Qiskit exactly as drawn and run with 1024 shots each on the ideal `AerSimulator` and on the IBM processor `ibm_marrakesh` (job `dar2ndrt55cs738sgol0`, physical qubits 153 and 154):

```
qc = QuantumCircuit(2, 2)
qc.x([0, 1])                # input |11>   (Figure 1 as given: qc.id(0) instead)
qc.h(1)                     # H on the MSB
qc.cx(1, 0)                 # CNOT, MSB controls
qc.measure([0, 1], [0, 1])  # bit string c1 c0, as in the composer
```

counts (1024 shots)	`00`	`01`	`10`	`11`
input $\|00\rangle$, simulator	521	0	0	503
input $\|00\rangle$, `ibm_marrakesh`	519	5	10	490
input $\|11\rangle$, simulator	0	503	521	0
input $\|11\rangle$, `ibm_marrakesh`	15	498	506	5



Both runs match the prediction. The simulator splits each Bell state between its two strings within the expected statistical fluctuation of $\sqrt{1024 \cdot \frac{1}{2} \cdot \frac{1}{2}} = 16$ shots. On the hardware, 98.5% (input $|00\rangle$) and 98.0% (input $|11\rangle$) of the shots land on the two correct strings; the remaining 1.5–2% are gate and readout errors. The modified circuit therefore outputs `01`/`10` at 50% each, i.e. $|\psi^-\rangle$ – up to its minus sign, which a histogram cannot see (see the remark below). The full code and output are in the notebook (**Appendix 1**).

> **Remark (A first attempt that went wrong).** The same five circuits of Problems 10–12 were first run on `ibm_kingston` with the default qubit choice, physical qubits 0 and 1 (job `dar2lobt55cs738sgmo0`). Only 67% and 57% of the shots landed on the correct strings here, and 50%, 38%, and 40% in Problems 11 and 12 – even though the exact circuits the device received simulate perfectly, and that pair's published calibration (CZ error 0.08%, readout error 1–2%) promised far better. Calibration data are snapshots taken hours earlier, and a qubit pair can drift in between. Re-running on the best-calibrated pair of `ibm_marrakesh` gave the clean results reported here. On real hardware, *which* physical qubits run the circuit matters as much as the circuit itself.

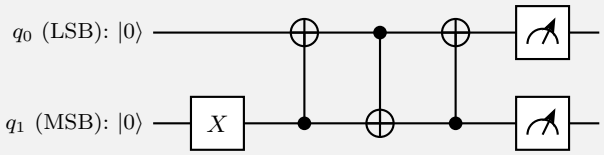
> **Remark.** A computational-basis histogram cannot see the minus sign: $|\psi^+\rangle$ (input $|01\rangle$) gives exactly the same two bars as $|\psi^-\rangle$. To confirm the output really is $|\psi^-\rangle$, undo the transformation before measuring (a CNOT and then H on q_1), which returns $|11\rangle$ with certainty.

Problem 11. Quantum Computing Exercise 2

Run the SWAP circuit in **Figure 2**, on IBM-Q using real hardware and simulation. Do necessary modification to change the input to $|10\rangle$ and check the output.

Solution

Setting up the input. Figure 3 shows the SWAP circuit of **Figure 2** built in IBM-Q: a NOT on q_0 prepares $|q_1 q_0\rangle = |01\rangle$, followed by CNOTs controlled by q_1, q_0, q_1 – that is, C_M, C_L, C_M of **Problem 3**. To feed in $|10\rangle$ instead, move the NOT gate from q_0 to q_1, the MSB:



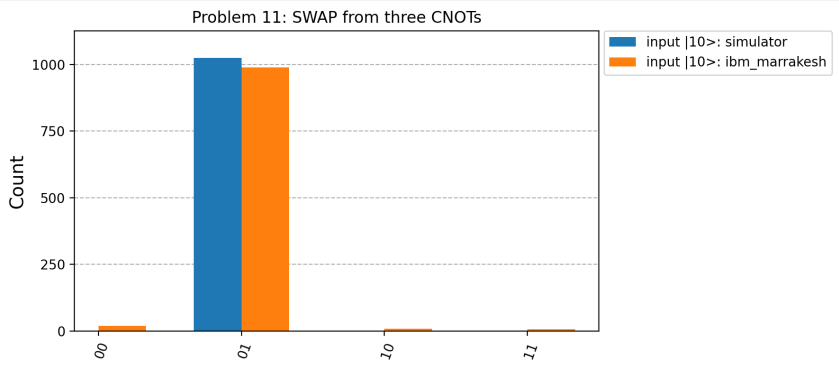
Expected output. Following the bits $(a, b) = (1, 0)$ (MSB first):

$$|10\rangle \xrightarrow{C_M} |1,\ 1 \oplus 0\rangle = |11\rangle \xrightarrow{C_L} |1 \oplus 1,\ 1\rangle = |01\rangle \xrightarrow{C_M} |0,\ 0 \oplus 1\rangle = |01\rangle\,,$$

so the output is $U_{\text{SWAP}}\,|10\rangle = |01\rangle$. It is a basis state, so it is measured with certainty: P(`01`) = 100%, read in the composer's $c_1 c_0$ bit string as `01`.

Running it. Run with 1024 shots on the `AerSimulator` and on `ibm_marrakesh` (same job and qubits as **Problem 10**):

counts (1024 shots)	`00`	`01`	`10`	`11`
simulator	0	1024	0	0
`ibm_marrakesh`	19	989	9	7



The simulator puts every shot on `01`, and the hardware puts 989 of 1024 (96.6%) there – the SWAP works as predicted. The largest error bar is `00` (19 shots): the $|1\rangle$ that ends up in q_0 relaxing back

to $|0\rangle$, the same bias discussed in **Problem 12**. A handy sanity check is that the result mirrors **Figure 3**: there input $|01\rangle$ gives `10`; here input $|10\rangle$ gives `01`.

Problem 12. Quantum Computing Exercise 3

Run the SWAP circuit in **Figure 2** on IBM-Q using real hardware and simulation but use the SWAP gate directly instead of using CNOT gates. Compare the outputs to **Figure 3**

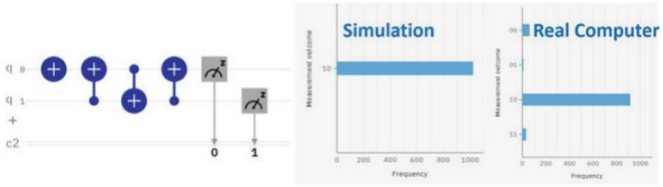
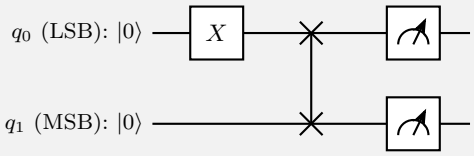


Figure 3: The quantum circuit in **Figure 1** built in IBM-Q. The simulation result is shown in the middle, and the real quantum computer execution result is shown on the right.

Solution

The circuit. Keep the input of **Figure 3** (a NOT on q_0, so $|q_1 q_0\rangle = |01\rangle$) and replace the three CNOTs with the composer's built-in SWAP gate:



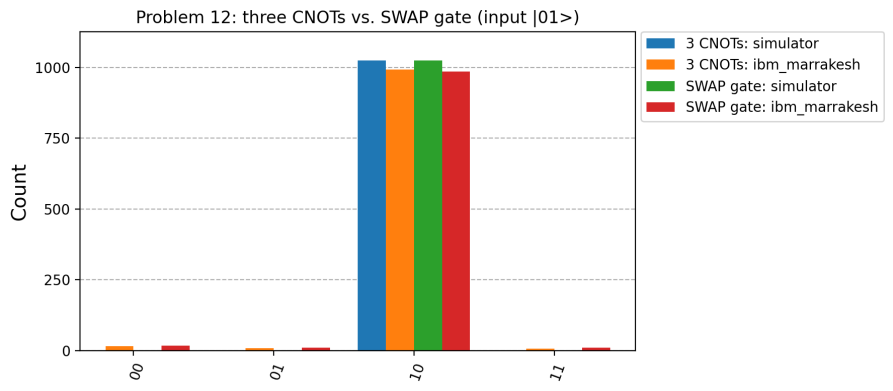
Simulation: identical to Figure 3. Problem 3 proved $C_M C_L C_M = U_{\text{SWAP}}$ as matrices, so the two circuits implement the same unitary and produce the same output state,

$$U_{\text{SWAP}}\,|01\rangle = |10\rangle = C_M C_L C_M\,|01\rangle\,,$$

which is measured as `10` with probability 100% – the single bar of the simulation in **Figure 3**. The notebook (**Appendix 1**) runs both versions and finds identical state vectors.

Running it. Both versions – the three CNOTs of **Figure 3** and the direct SWAP gate – were run side by side with 1024 shots each on the `AerSimulator` and on `ibm_marrakesh` (same job and qubits as **Problem 10**):

counts (1024 shots)	`00`	`01`	`10`	`11`
three CNOTs, simulator	0	0	1024	0
three CNOTs, `ibm_marrakesh`	16	8	993	7
SWAP gate, simulator	0	0	1024	0
SWAP gate, `ibm_marrakesh`	18	10	986	10



Comparison with Figure 3.

- **Simulation:** identical – every shot on `10`, for both versions and in **Figure 3**.
- **Real hardware:** the SWAP gate puts 96.3% of the shots on `10` and the three CNOTs 97.0%. The 7-shot difference is within one standard deviation of shot noise (about 8 shots), so the two are statistically the same. Both beat the roughly 90% of the textbook's hardware run in **Figure 3**.
- **Why they match:** IBM processors have no native SWAP. Transpiling for `ibm_marrakesh` (optimization level 1, as used for the run) turns the SWAP gate into *three* CZ gates – exactly as many as the three-CNOT version – so the device executes essentially the circuit of **Figure 3** either way.

At optimization level 3 the transpiler goes further and removes this SWAP entirely (0 CZ gates), because a SWAP right before measurement can be replaced by relabelling which qubit feeds which classical bit. The three-CNOT version still costs three CZ gates there, as the notebook (**Appendix 1**) shows.

> **Remark.** The largest wrong outcome is `00` in both of our runs (16 and 18 shots) and in **Figure 3**: the correct result `10` with q_1 fallen back to $|0\rangle$. That pattern is consistent with energy relaxation (T_1 decay, $|1\rangle \to |0\rangle$) during the gates and readout, the dominant error on superconducting qubits – errors push outcomes toward `0`s, whichever way the SWAP is built.

APPENDIX 1

Assignment 07 | Code Solution

assignment-07-code

September 25, 2026

1 Assignment 07

1.1 Import External Libraries

[1]:
```
import numpy as np
import sympy as sp
import matplotlib.pyplot as plt

from IPython.display import display, Math

from qiskit import QuantumCircuit
from qiskit.transpiler.preset_passmanagers import generate_preset_pass_manager
from qiskit.visualization import plot_histogram
from qiskit_aer import AerSimulator
from qiskit_ibm_runtime import QiskitRuntimeService
```

1.2 1. Z-Gate

[2]:
```
# --- Problem 1: the Z-gate turns |-> into |+> ---
def latex_matrix(M):
    """LaTeX for a numeric array: rounded to 4 d.p., whole numbers shown as integers."""
    M = np.round(np.real_if_close(np.asarray(M, dtype=complex)), 4) + 0.0
    return sp.latex(sp.Matrix(M).applyfunc(lambda v: sp.Integer(int(v)) if float(v).is_integer()␣
  ↪else v))


Z     = np.array([[1, 0], [0, -1]])
plus  = np.array([1,  1]) / np.sqrt(2)
minus = np.array([1, -1]) / np.sqrt(2)

display(Math(r"Z\ket{-} = " + latex_matrix(Z @ minus)))
print("Problem 1  --  Z|-> == |+> ?", np.allclose(Z @ minus, plus))
print("Problem 1  --  Z|+> == |-> ?", np.allclose(Z @ plus, minus))
```

$$Z\left|-\right\rangle = \begin{bmatrix}0.7071\\0.7071\end{bmatrix}$$

```
Problem 1  --  Z|-> == |+> ? True
Problem 1  --  Z|+> == |-> ? True
```

1.3 2. Controlled-Z Gate

[3]:
```
# --- Problem 2: controlled-Z from the controlled phase shift, and CZ as a CNOT in the +/- basis␣
  ↪---
I2 = np.eye(2)
X  = np.array([[0, 1], [1, 0]])
```

```
H  = np.array([[1, 1], [1, -1]]) / np.sqrt(2)
P0 = np.array([[1, 0], [0, 0]])   # |0><0|
P1 = np.array([[0, 0], [0, 1]])   # |1><1|


def CP(phi):
    """Controlled phase shift with the MSB as control: |0><0| (x) I + |1><1| (x) P(phi)."""
    return np.kron(P0, I2) + np.kron(P1, np.diag([1, np.exp(1j * phi)]))


CZ   = CP(np.pi)
CNOT = np.kron(P0, I2) + np.kron(P1, X)

display(Math(r"CP(\pi) = " + latex_matrix(CZ)))
print("Problem 2  --  CP(pi) == diag(1, 1, 1, -1) ?", np.allclose(CZ, np.diag([1, 1, 1, -1])))

print("---------------")

# target in the |+>/|-> basis: change basis with I (x) H  (H|0> = |+>, H|1> = |->)
CZ_pm = np.kron(I2, H) @ CZ @ np.kron(I2, H)
display(Math(r"(I \otimes H)\,CZ\,(I \otimes H) = " + latex_matrix(CZ_pm)))
print("Problem 2  --  equals CNOT ?", np.allclose(CZ_pm, CNOT))
print("Problem 2  --  CZ|1+> == |1-> ?", np.allclose(CZ @ np.kron([0, 1], plus), np.kron([0, 1],␣
 ↪minus)))
```

$$CP(\pi) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{bmatrix}$$

```
Problem 2  --  CP(pi) == diag(1, 1, 1, -1) ? True
---------------
```

$$(I \otimes H)\,CZ\,(I \otimes H) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

```
Problem 2  --  equals CNOT ? True
Problem 2  --  CZ|1+> == |1-> ? True
```

1.4 3. Implementation of a SWAP gate

[4]:
```
# --- Problem 3: SWAP from CNOT (MSB ctrl), CNOT (LSB ctrl), CNOT (MSB ctrl) ---
def basis(bits):
    """|b1 b0> as a length-4 vector, MSB first."""
    v = np.zeros(4)
    v[int(bits, 2)] = 1
    return v


def gate_from_map(f):
    """Matrix whose column |j> is f(|j>), i.e. built from the gate's truth table."""
    return np.column_stack([basis(f(format(j, "02b"))) for j in range(4)])


C_M  = gate_from_map(lambda ab: ab[0] + str(int(ab[0]) ^ int(ab[1])))   # |a b> -> |a, a xor b>
C_L  = gate_from_map(lambda ab: str(int(ab[0]) ^ int(ab[1])) + ab[1])   # |a b> -> |a xor b, b>
SWAP = gate_from_map(lambda ab: ab[::-1])                               # |a b> -> |b a>

display(Math(r"C_L = " + latex_matrix(C_L)))
```

```
display(Math(r"C_M\,C_L\,C_M = " + latex_matrix(C_M @ C_L @ C_M)))
print("Problem 3  --  C_M equals the CNOT of Problem 2 ?", np.allclose(C_M, CNOT))
print("Problem 3  --  C_M C_L C_M == SWAP ?", np.allclose(C_M @ C_L @ C_M, SWAP))
```

$$C_L = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

$$C_M\, C_L\, C_M = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

```
Problem 3  --  C_M equals the CNOT of Problem 2 ? True
Problem 3  --  C_M C_L C_M == SWAP ? True
```

1.5 4. 1-qubit Hadamard Gate

[5]:
```
# --- Problem 4: H (alpha|0> + beta|1>) = alpha|+> + beta|-> ---
alpha, beta = sp.symbols("alpha beta")
H_sym     = sp.Matrix([[1, 1], [1, -1]]) / sp.sqrt(2)
plus_sym  = sp.Matrix([1,  1]) / sp.sqrt(2)
minus_sym = sp.Matrix([1, -1]) / sp.sqrt(2)

lhs = H_sym * sp.Matrix([alpha, beta])
rhs = alpha * plus_sym + beta * minus_sym

display(Math(r"H\ket{\psi} = " + sp.latex(lhs)))
display(Math(r"\alpha\ket{+} + \beta\ket{-} = " + sp.latex(sp.simplify(rhs))))
print("Problem 4  --  equal?", sp.simplify(lhs - rhs) == sp.zeros(2, 1))
```

$$H\left|\psi\right\rangle = \begin{bmatrix} \frac{\sqrt{2}\alpha}{2} + \frac{\sqrt{2}\beta}{2} \\ \frac{\sqrt{2}\alpha}{2} - \frac{\sqrt{2}\beta}{2} \end{bmatrix}$$

$$\alpha\left|+\right\rangle + \beta\left|-\right\rangle = \begin{bmatrix} \frac{\sqrt{2}(\alpha+\beta)}{2} \\ \frac{\sqrt{2}(\alpha-\beta)}{2} \end{bmatrix}$$

```
Problem 4  --  equal? True
```

1.6 5–7. n-qubit Hadamard Gate (Problems 5–7)

[6]:
```
# --- Problems 5-7: H^{(x)2}|2> three ways ---
n, y = 2, 2


def dot_mod2(x, y):
    """Bitwise inner product x . y taken mod 2."""
    return bin(x & y).count("1") % 2


# Problem 5: the sum formula  (1 / 2^{n/2}) sum_x (-1)^{x.y} |x>
formula = np.array([(-1) ** dot_mod2(x, y) for x in range(2 ** n)]) / 2 ** (n / 2)

# Problem 6: the expansion  (H|y1>) (x) (H|y0>)
ket = [np.array([1, 0]), np.array([0, 1])]
y1, y0 = divmod(y, 2)
expansion = np.kron(H @ ket[y1], H @ ket[y0])
```

```
# Problem 7: matrix multiplication  (H (x) H)|2>
display(Math(r"H \otimes H = " + latex_matrix(np.kron(H, H))))
matrix = np.kron(H, H) @ basis(format(y, "02b"))

print("n =", n)
for name, vec in [("Problem 5 (formula)", formula),
                  ("Problem 6 (expansion)", expansion),
                  ("Problem 7 (matrix)", matrix)]:
    display(Math(r"\text{" + name + r"}: \; H^{\otimes 2}\ket{2} = " + latex_matrix(vec.
  ↪reshape(1, -1))))
print("Problems 5-7  --  all equal?", np.allclose(formula, expansion) and np.allclose(expansion,␣
  ↪matrix))
```

$$H \otimes H = \begin{bmatrix} 0.5 & 0.5 & 0.5 & 0.5 \\ 0.5 & -0.5 & 0.5 & -0.5 \\ 0.5 & 0.5 & -0.5 & -0.5 \\ 0.5 & -0.5 & -0.5 & 0.5 \end{bmatrix}$$

```
n = 2
```

$\text{Problem 5 (formula)}: \; H^{\otimes 2}\ket{2} = \begin{bmatrix} 0.5 & 0.5 & -0.5 & -0.5 \end{bmatrix}$

$\text{Problem 6 (expansion)}: \; H^{\otimes 2}\ket{2} = \begin{bmatrix} 0.5 & 0.5 & -0.5 & -0.5 \end{bmatrix}$

$\text{Problem 7 (matrix)}: \; H^{\otimes 2}\ket{2} = \begin{bmatrix} 0.5 & 0.5 & -0.5 & -0.5 \end{bmatrix}$

```
Problems 5-7  --  all equal? True
```

1.7 8. SWAP Gate Implementation

[7]:
```
# --- Problem 8: SWAP from CNOT (LSB ctrl), CNOT (MSB ctrl), CNOT (LSB ctrl) ---
display(Math(r"C_M\,C_L = " + latex_matrix(C_M @ C_L)))
display(Math(r"C_L\,C_M\,C_L = " + latex_matrix(C_L @ C_M @ C_L)))
print("Problem 8  --  C_L C_M C_L == SWAP ?", np.allclose(C_L @ C_M @ C_L, SWAP))

print("---------------")

# intuition method: follow every basis state through the three gates
for j in range(4):
    v = basis(format(j, "02b"))
    path = [v, C_L @ v, C_M @ C_L @ v, C_L @ C_M @ C_L @ v]
    print("  ->  ".join(f"|{format(int(np.argmax(s)), '02b')}>" for s in path))
```

$$C_M\, C_L = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

$$C_L\, C_M\, C_L = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

```
Problem 8  --  C_L C_M C_L == SWAP ? True
---------------
|00>  ->  |00>  ->  |00>  ->  |00>
|01>  ->  |11>  ->  |10>  ->  |10>
|10>  ->  |10>  ->  |11>  ->  |01>
|11>  ->  |01>  ->  |01>  ->  |11>
```

1.8 9. LSB-controlling CNOT Gate

[8]:
```
# --- Problem 9: derive the LSB-controlling CNOT from its definition ---
# U = sum_{a,b} |a xor b, b><a b|   (one outer product |output><input| per truth-table row)
U_center = sum(np.outer(basis(f"{a ^ b}{b}"), basis(f"{a}{b}")) for a in (0, 1) for b in (0, 1))
display(Math(r"U_{\text{XOR},\text{center}} = " + latex_matrix(U_center)))
print("Problem 9  --  equals C_L ?", np.allclose(U_center, C_L))
print("Problem 9  --  equals I (x) |0><0| + X (x) |1><1| ?", np.allclose(U_center, np.kron(I2,␣
  ↪P0) + np.kron(X, P1)))
print("Problem 9  --  equals SWAP C_M SWAP ?", np.allclose(U_center, SWAP @ C_M @ SWAP))

print("---------------")

U_left = U_right = C_M
display(Math(r"U_{\text{XOR},\text{right}}\,U_{\text{XOR},\text{center}} = " +␣
  ↪latex_matrix(U_right @ U_center)))
print("Problem 9  --  U_right U_center U_left == SWAP ?", np.allclose(U_right @ U_center @␣
  ↪U_left, SWAP))
```

$$U_{\text{XOR,center}} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

```
Problem 9  --  equals C_L ? True
Problem 9  --  equals I (x) |0><0| + X (x) |1><1| ? True
Problem 9  --  equals SWAP C_M SWAP ? True
---------------
```

$$U_{\text{XOR,right}}\,U_{\text{XOR,center}} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

```
Problem 9  --  U_right U_center U_left == SWAP ? True
```

1.9 10. Quantum Computing Exercise 1

[9]:
```
# --- Problem 10: the basis-transformation circuit of Figure 1, then with input |11> ---
SHOTS = 1024
rng = np.random.default_rng(2110515)


def run(gates, label, shots=SHOTS):
    """Apply the gates (in time order) to |00>; print exact probabilities and sampled counts."""
    state = basis("00")
    for g in gates:
        state = g @ state
    probs = np.abs(state) ** 2
    counts = rng.multinomial(shots, probs / probs.sum())
    display(Math(label + " = " + latex_matrix(state.reshape(1, -1))))
    for k in range(4):
        print(f"  {format(k, '02b')}:  P = {probs[k]:7.2%}   simulated counts = {counts[k]:4d} /␣
  ↪{shots}")
    return state


X_M, X_L, H_M = np.kron(X, I2), np.kron(I2, X), np.kron(H, I2)   # MSB is the first tensor factor

print("Figure 1 as given (input |00>; the I gate on q0 does nothing):")
run([H_M, C_M], r"(\text{CNOT})(H \otimes I)\ket{00}")
```

```
print("---------------")

print("Modified: X on both qubits first (input |11>):")
out = run([X_M, X_L, H_M, C_M], r"(\text{CNOT})(H \otimes I)\ket{11}")
psi_minus = (basis("01") - basis("10")) / np.sqrt(2)
print("Problem 10  --  output == |psi^-> ?", np.allclose(out, psi_minus))
```

Figure 1 as given (input |00>; the I gate on q0 does nothing):

$$(\mathrm{CNOT})(H \otimes I)\,|00\rangle = [0.7071 \quad 0 \quad 0 \quad 0.7071]$$

```
  00:  P =  50.00%   simulated counts =  536 / 1024
  01:  P =   0.00%   simulated counts =    0 / 1024
  10:  P =   0.00%   simulated counts =    0 / 1024
  11:  P =  50.00%   simulated counts =  488 / 1024
---------------
Modified: X on both qubits first (input |11>):
```

$$(\mathrm{CNOT})(H \otimes I)\,|11\rangle = [0 \quad 0.7071 \quad -0.7071 \quad 0]$$

```
  00:  P =   0.00%   simulated counts =    0 / 1024
  01:  P =  50.00%   simulated counts =  507 / 1024
  10:  P =  50.00%   simulated counts =  517 / 1024
  11:  P =   0.00%   simulated counts =    0 / 1024
Problem 10  --  output == |psi^-> ? True
```

1.10 11. Quantum Computing Exercise 2

[10]:
```
# --- Problem 11: the SWAP circuit of Figure 2 with input |10> (X on the MSB) ---
out = run([X_M, C_M, C_L, C_M], r"C_M\,C_L\,C_M\ket{10}")
print("Problem 11  --  output == |01> ?", np.allclose(out, basis("01")))
```

$$C_M\,C_L\,C_M\,|10\rangle = [0 \quad 1 \quad 0 \quad 0]$$

```
  00:  P =   0.00%   simulated counts =    0 / 1024
  01:  P = 100.00%   simulated counts = 1024 / 1024
  10:  P =   0.00%   simulated counts =    0 / 1024
  11:  P =   0.00%   simulated counts =    0 / 1024
Problem 11  --  output == |01> ? True
```

1.11 12. Quantum Computing Exercise 3

[11]:
```
# --- Problem 12: input |01> as in Figure 3 -- three CNOTs vs. the SWAP gate directly ---
print("Three CNOTs (Figure 3):")
out_cnot = run([X_L, C_M, C_L, C_M], r"C_M\,C_L\,C_M\ket{01}")

print("---------------")

print("SWAP gate directly:")
out_swap = run([X_L, SWAP], r"U_{\text{SWAP}}\ket{01}")
print("Problem 12  --  identical outputs?", np.allclose(out_cnot, out_swap))
```

Three CNOTs (Figure 3):

$$C_M\,C_L\,C_M\,|01\rangle = [0 \quad 0 \quad 1 \quad 0]$$

```
  00:  P =   0.00%   simulated counts =    0 / 1024
  01:  P =   0.00%   simulated counts =    0 / 1024
  10:  P = 100.00%   simulated counts = 1024 / 1024
  11:  P =   0.00%   simulated counts =    0 / 1024
```

```
---------------
SWAP gate directly:
```

$U_{\mathrm{SWAP}}\,|01\rangle = [0 \quad 0 \quad 1 \quad 0]$

```
  00:  P =   0.00%   simulated counts =    0 / 1024
  01:  P =   0.00%   simulated counts =    0 / 1024
  10:  P = 100.00%   simulated counts = 1024 / 1024
  11:  P =   0.00%   simulated counts =    0 / 1024
Problem 12  --  identical outputs? True
```

1.12 10–12. Running on IBM Quantum (Problems 10–12)

[12]:
```python
# --- Problems 10-12 on IBM Quantum: AerSimulator vs. real hardware ---
# Each hardware job was submitted once (one Sampler job holding all five circuits, 1024 shots␣
  ↪each).
# Re-running this notebook only *fetches* the results, so no further QPU time is spent.
HW_JOB_ID    = "dar2ndrt55cs738sgol0"   # ibm_marrakesh, pinned to the best-calibrated pair (153,␣
  ↪154)
FIRST_JOB_ID = "dar2lobt55cs738sgmo0"   # first attempt: ibm_kingston, default layout (0, 1) --␣
  ↪see the last cell
SHOTS = 1024


def figure1(prepare_11):
    """Figure 1: H on the MSB (q1), then CNOT q1 -> q0. prepare_11 turns the I on q0 into X and␣
  ↪adds X on q1."""
    qc = QuantumCircuit(2, 2)
    if prepare_11:
        qc.x([0, 1])
    else:
        qc.id(0)
    qc.h(1)
    qc.cx(1, 0)
    qc.measure([0, 1], [0, 1])
    return qc


def swap_circuit(input_q1q0, direct):
    """Figures 2/3: prepare |q1 q0>, then SWAP as three CNOTs (C_M, C_L, C_M) or as one SWAP gate.
  ↪"""
    qc = QuantumCircuit(2, 2)
    if input_q1q0[0] == "1":
        qc.x(1)
    if input_q1q0[1] == "1":
        qc.x(0)
    if direct:
        qc.swap(0, 1)
    else:
        qc.cx(1, 0)
        qc.cx(0, 1)
        qc.cx(1, 0)
    qc.measure([0, 1], [0, 1])
    return qc


# same order as the PUBs of the submitted job
CIRCUITS = {
    "p10_input00": figure1(prepare_11=False),
    "p10_input11": figure1(prepare_11=True),
```

```
    "p11_cnot_input10": swap_circuit("10", direct=False),
    "p12_cnot_input01": swap_circuit("01", direct=False),
    "p12_swap_input01": swap_circuit("01", direct=True),
}

sim = AerSimulator(seed_simulator=2110515)
sim_counts = {name: sim.run(qc, shots=SHOTS).result().get_counts() for name, qc in CIRCUITS.
  ↪items()}

service = QiskitRuntimeService()   # saved default account (set up in Lab 00)
hw_job = service.job(HW_JOB_ID)
hw_result = hw_job.result()
hw_counts = {name: hw_result[i].data.c.get_counts() for i, name in enumerate(CIRCUITS)}
backend_name = hw_job.backend().name
print(f"Hardware job {HW_JOB_ID} on {backend_name}: {hw_job.status()}")

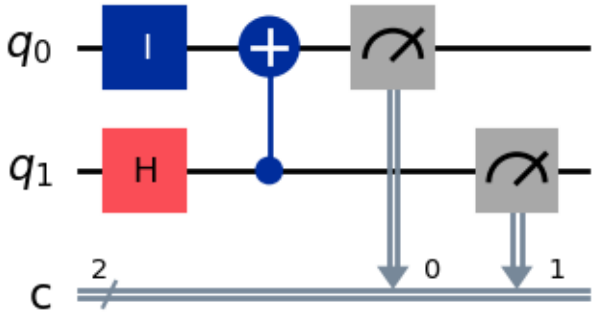

def draw(name):
    fig = CIRCUITS[name].draw("mpl")
    display(fig)
    plt.close(fig)

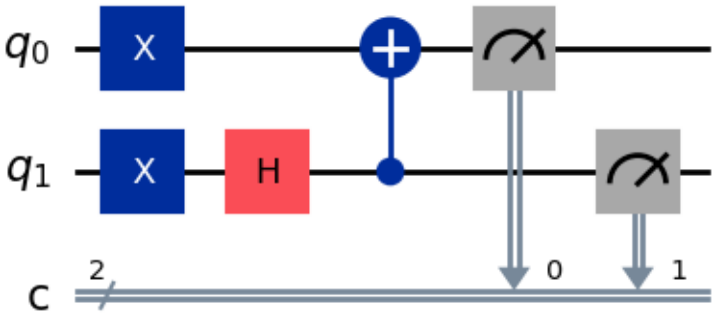

def report(name, label):
    """Counts per bit string c1c0 (the composer's order): simulator vs. hardware."""
    print(label)
    for k in ("00", "01", "10", "11"):
        s, h = sim_counts[name].get(k, 0), hw_counts[name].get(k, 0)
        print(f"  {k}:  simulator {s:4d}   {backend_name} {h:4d}  ({h / SHOTS:6.2%})")


def histogram(names, labels, title, filename):
    """Grouped simulator/hardware histogram, also saved to ../images for the report."""
    data = [counts[n] for n in names for counts in (sim_counts, hw_counts)]
    legend = [f"{lbl}: {src}" for lbl in labels for src in ("simulator", backend_name)]
    fig = plot_histogram(data, legend=legend, title=title, figsize=(9, 4), bar_labels=False)
    fig.savefig(f"../images/{filename}", dpi=200, bbox_inches="tight")
    display(fig)
    plt.close(fig)
```

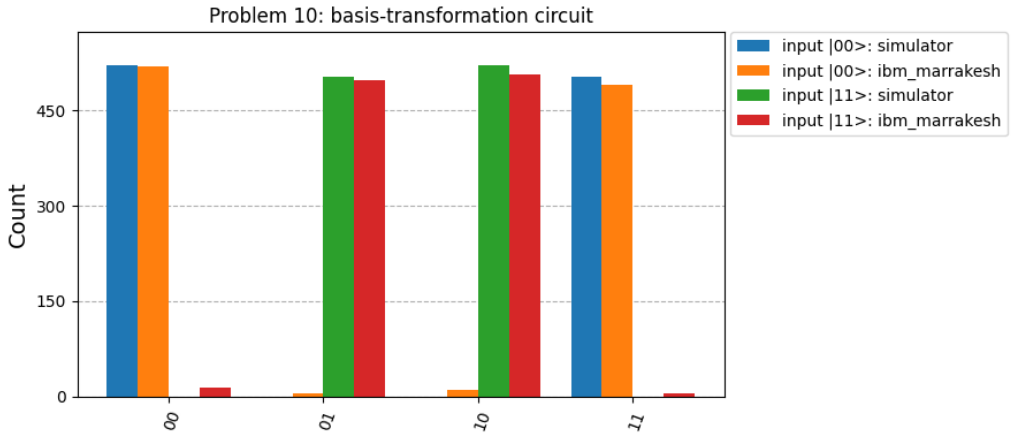
```
Hardware job dar2ndrt55cs738sgol0 on ibm_marrakesh: DONE
```

[13]:
```
# --- Problem 10: Figure 1 as given (input |00>) and with input |11> ---
draw("p10_input00")
draw("p10_input11")
report("p10_input00", "Figure 1, input |00>  (expected |phi+>: 00 / 11)")
report("p10_input11", "Figure 1, input |11>  (expected |psi->: 01 / 10)")
histogram(["p10_input00", "p10_input11"], ["input |00>", "input |11>"],
          "Problem 10: basis-transformation circuit", "problem-10-answer.png")
```

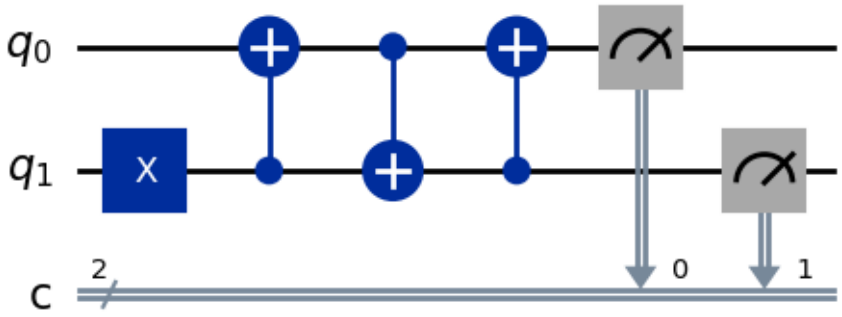




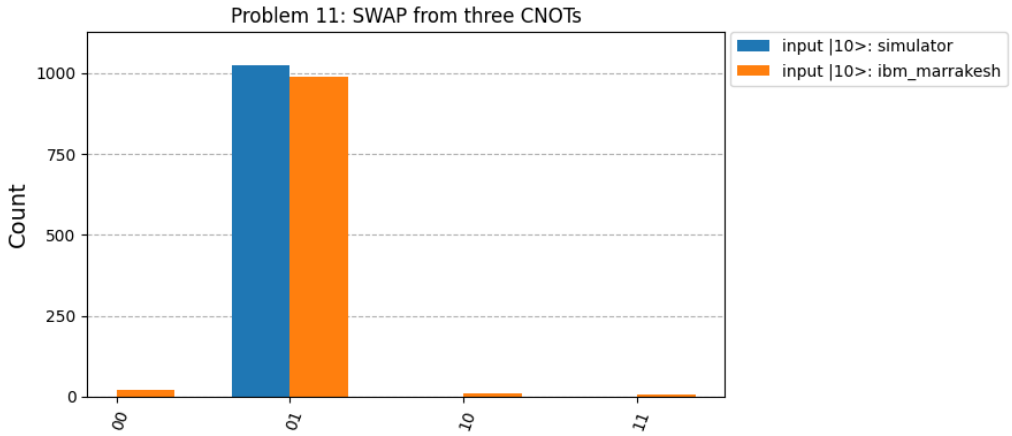

```
Figure 1, input |00>  (expected |phi+>: 00 / 11)
  00:  simulator  521   ibm_marrakesh  519  (50.68%)
  01:  simulator    0   ibm_marrakesh    5  ( 0.49%)
  10:  simulator    0   ibm_marrakesh   10  ( 0.98%)
  11:  simulator  503   ibm_marrakesh  490  (47.85%)
Figure 1, input |11>  (expected |psi->: 01 / 10)
  00:  simulator    0   ibm_marrakesh   15  ( 1.46%)
  01:  simulator  503   ibm_marrakesh  498  (48.63%)
  10:  simulator  521   ibm_marrakesh  506  (49.41%)
  11:  simulator    0   ibm_marrakesh    5  ( 0.49%)
```



[14]:
```
# --- Problem 11: SWAP circuit (three CNOTs) with input |10> ---
draw("p11_cnot_input10")
report("p11_cnot_input10", "SWAP (3 CNOTs), input |10>  (expected 01)")
histogram(["p11_cnot_input10"], ["input |10>"],
          "Problem 11: SWAP from three CNOTs", "problem-11-answer.png")
```



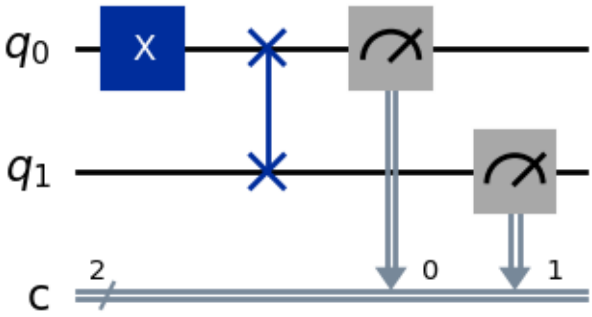

```
SWAP (3 CNOTs), input |10>  (expected 01)
  00:  simulator    0   ibm_marrakesh   19  ( 1.86%)
  01:  simulator 1024   ibm_marrakesh  989  (96.58%)
  10:  simulator    0   ibm_marrakesh    9  ( 0.88%)
  11:  simulator    0   ibm_marrakesh    7  ( 0.68%)
```



[15]:
```
# --- Problem 12: input |01> -- three CNOTs (Figure 3) vs. the SWAP gate directly ---
draw("p12_swap_input01")
report("p12_cnot_input01", "Three CNOTs, input |01>  (expected 10)")
report("p12_swap_input01", "SWAP gate,   input |01>  (expected 10)")
histogram(["p12_cnot_input01", "p12_swap_input01"], ["3 CNOTs", "SWAP gate"],
          "Problem 12: three CNOTs vs. SWAP gate (input |01>)", "problem-12-answer.png")

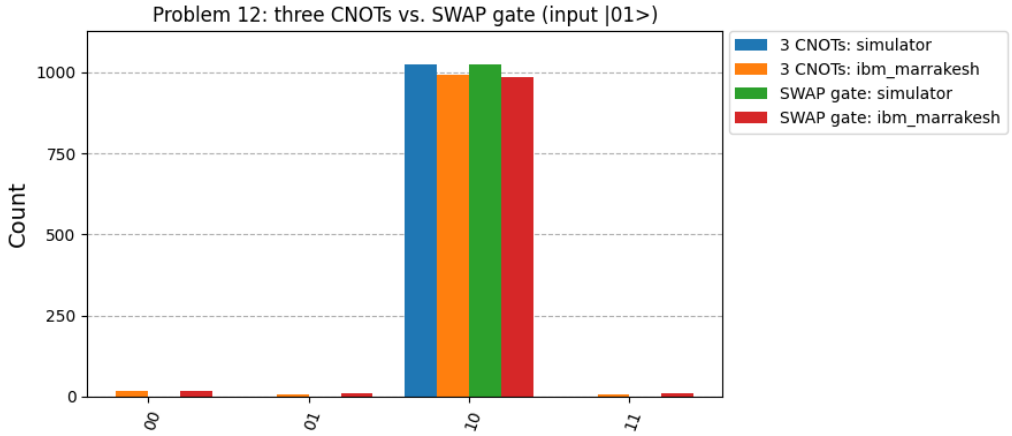
print("---------------")

# what the transpiler actually runs on the device (native two-qubit gate: CZ)
backend = service.backend(backend_name)
for level in (1, 3):
    pm = generate_preset_pass_manager(optimization_level=level, backend=backend,␣
 ↪seed_transpiler=7)
    for name in ("p12_cnot_input01", "p12_swap_input01"):
        print(f"  optimization level {level}, {name}: {pm.run(CIRCUITS[name]).count_ops().
 ↪get('cz', 0)} CZ gate(s)")
```




```
Three CNOTs, input |01>  (expected 10)
  00:  simulator    0   ibm_marrakesh   16  ( 1.56%)
  01:  simulator    0   ibm_marrakesh    8  ( 0.78%)
  10:  simulator 1024   ibm_marrakesh  993  (96.97%)
  11:  simulator    0   ibm_marrakesh    7  ( 0.68%)
SWAP gate,   input |01>  (expected 10)
```

```
  00:  simulator    0   ibm_marrakesh   18  ( 1.76%)
  01:  simulator    0   ibm_marrakesh   10  ( 0.98%)
  10:  simulator 1024   ibm_marrakesh  986  (96.29%)
  11:  simulator    0   ibm_marrakesh   10  ( 0.98%)
```



```
  optimization level 1, p12_cnot_input01: 3 CZ gate(s)
  optimization level 1, p12_swap_input01: 3 CZ gate(s)
  optimization level 3, p12_cnot_input01: 3 CZ gate(s)
  optimization level 3, p12_swap_input01: 0 CZ gate(s)
```

[16]:
```
# --- The first hardware attempt (ibm_kingston, physical qubits 0 and 1) ---
# Same five circuits, default layout. The circuits it ran simulate perfectly, yet the device␣
  ↪returned
# far noisier counts than its calibration data predicts, so the job was re-run on the␣
  ↪best-calibrated
# qubit pair (the results used above). Both runs are kept for comparison.
EXPECTED = {
    "p10_input00": ("00", "11"),
    "p10_input11": ("01", "10"),
    "p11_cnot_input10": ("01",),
    "p12_cnot_input01": ("10",),
    "p12_swap_input01": ("10",),
}
first_job = service.job(FIRST_JOB_ID)
first_result = first_job.result()
print(f"share of shots on the expected outcome(s):   {first_job.backend().name} (0, 1)   vs.  ␣
  ↪{backend_name}")
for i, name in enumerate(CIRCUITS):
    first = first_result[i].data.c.get_counts()
    ok_first = sum(first.get(k, 0) for k in EXPECTED[name]) / SHOTS
    ok_best = sum(hw_counts[name].get(k, 0) for k in EXPECTED[name]) / SHOTS
    print(f"  {name:18s}  {ok_first:7.1%}   vs.   {ok_best:6.1%}")
```

```
share of shots on the expected outcome(s):   ibm_kingston (0, 1)   vs.
ibm_marrakesh
  p10_input00           66.6%   vs.    98.5%
  p10_input11           56.9%   vs.    98.0%
  p11_cnot_input10      50.0%   vs.    96.6%
  p12_cnot_input01      37.8%   vs.    97.0%
  p12_swap_input01      39.8%   vs.    96.3%
```